Intermediate (build the loop yourself) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — Doc06's round trip, repeated
- Hint 2 — The plan, and almost the whole thing

Hint 1 — Doc06's round trip, repeated

Each turn of the loop has three parts — the names from the ReAct topic:

- **Think:** `response = model_with_tools.invoke(messages)`, then `messages.append(response)`.
- **Act:** if `response.tool_calls` is empty, the model is done — return `response.content`. Otherwise run every call with `tool.invoke(call["args"])`.
- **Observe:** append `{"role": "tool", "content": result, "tool_call_id": call["id"]}` for each call.

Keep a `steps` list (step number, tool, result), return it in an `AgentResult` dataclass, and when the loop runs out, raise `MaxIterationsExceeded(message, steps)` — a named error that carries the partial log, like Doc02's `PermanentHTTPError` carries its status code.

Hint 2 — The plan, and almost the whole thing

```
run_agent(task, max_iterations) -> AgentResult:
    for step in range(max_iterations):
        think; if done: return AgentResult(answer, steps)
        for each call:
            result, error = run_tool(call, ALL_TOOLS)
            observation = result, or "Error: " + error
            log it; add a tool message; add a step dict
    raise MaxIterationsExceeded(message, steps)

run_tool(call, tools) -> (result, error):
    find_tool (Doc06) -> None? return (None, "unknown tool")
```

```
    try: return (str(tool.invoke(args)), None)
    except Exception as e: return (None, str(e))
```

Write the loop first with `add` only, then widen it to every tool in `ALL_TOOLS`. Then add a factory `make_flaky_add(state)` that returns a new `@tool` failing on its first call; `run_tool_checked()` that builds the tool's Pydantic model from the arguments and calls `.invoke(checked.model_dump())`; and the same loop on the raw SDK (`client.chat.completions.create(..., tools=[schema])`, `json.loads(call.function.arguments)`). Compare against the Solution.

Full solution: Show me the solution